Machine Learning
Unit-III (B): K-Nearest Neighbor (KNN)

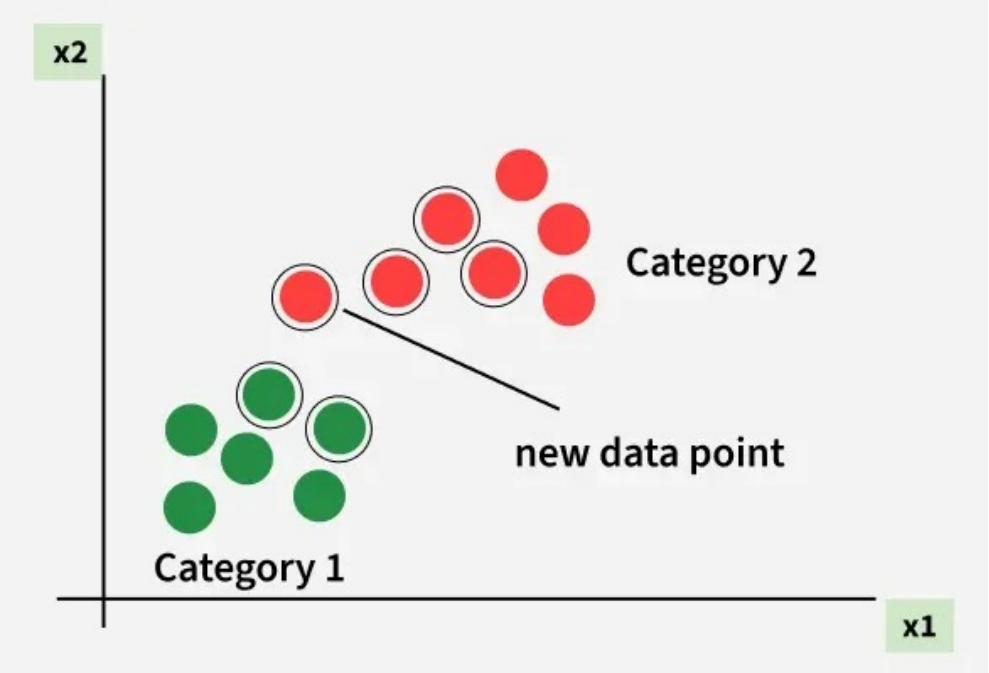
Dr. Satveer Singh
Assistant Professor,
JK Lakshmipat University, Jaipur.

K-Nearest Neighbor Algorithm

- KNN is a simple and widely used machine learning technique for classification and regression tasks.
- It works by identifying the K closest data points to a given input and making predictions based on the majority class or average value of those neighbors.
- Uses distance metrics like Euclidean distance to find nearest neighbors.
- Since KNN makes no assumptions about the underlying data distribution, it makes it a non-parametric and instance-based learning method.
- KNN is also called as a lazy learner algorithm because it does not learn from the training set immediately instead it stores the entire dataset and performs computations only at the time of classification.

Example

- Consider two features, i.e., Category 1 and Category 2.
- The image shows how KNN predicts the category of a new data point based on its closest neighbours.



Example

- The green points represent Category 1 and the red points represent Category 2.
- The new data point checks its closest neighbors (circled points) based on $K = 5$.
- Since the majority of its closest neighbors are red points (Category 2).
- KNN predicts the new data point belongs to Category 2.
- KNN works by using proximity and majority voting to make predictions.

What is $'K'$ in K Nearest Neighbour?

- In the KNN algorithm, K is just a number that tells the algorithm how many nearby points or neighbors to look at when it makes a decision.

Example

- Imagine you're deciding which fruit it is based on its shape and size. You compare it to fruits you already know.
- If $K = 3$, the algorithm looks at the 3 closest fruits to the new one.
- If 2 of those 3 fruits are apples and 1 is a banana, the algorithm says the new fruit is an apple because most of its neighbors are apples.

How to choose the value of *K* for KNN Algorithm?

- The value of *K* in KNN decides how many neighbors the algorithm looks at when making a prediction.
- Choosing the right *K* is important for good results.
- If the data has lots of noise or outliers, using a larger *K* can make the predictions more stable.
- But if *K* is too large the model may become too simple and miss important patterns and this is called underfitting.
- So *K* should be picked carefully based on the data.

Statistical Methods for Selecting K

Cross-Validation

- Cross-Validation is a good way to find the best value of K is by using K-fold cross-validation.
- This means dividing the dataset into K parts.
- The model is trained on some of these parts and tested on the remaining ones.
- This process is repeated for each part.
- The K value that gives the highest average accuracy during these tests is usually the best one to use.

Statistical Methods for Selecting K

Elbow Method

- In Elbow Method, we draw a graph showing the error rate or accuracy for different K values.
- As K increases the error usually drops at first.
- But after a certain point error stops decreasing quickly.
- The point where the curve changes direction and looks like an "elbow" is usually the best choice for K.

Statistical Methods for Selecting *K*

Odd Values for k

- It's a good idea to use an odd number for *K* especially in classification problems.
- This helps avoid ties when deciding which class is the most common among the neighbors.

Tie Condition in KNN

- In KNN, a tie condition occurs when two or more classes receive the same number of votes among the K nearest neighbors.
- This is common when:
  - K is even (e.g., $K = 4$)
  - Data is imbalanced or overlapping
- There's no single "correct" way, several standard strategies are used depending on the situation.

Use an Odd Value of K (Simplest Fix)

- Choose $K = 3, 5, 7, \ldots$
- Reduces the chance of ties (especially in binary classification).
- This is the most commonly recommended approach.

Distance-Based Weighting (Weighted KNN)

- Instead of equal voting, assign higher weight to closer neighbors:

$$\text{weight}_i = \frac{1}{d(x, x_i)}$$

- Closer neighbors contribute more.
- Compute weighted votes for each class.
- Choose class with highest total weight.
- Very effective in practice.

Choose Class with Minimum Total Distance

- For each tied class:
  - Sum distances of neighbors belonging to that class.
  - Select the class with smaller total distance.
- Intuition: closer cluster should dominate.

Distance Metrics Used in KNN Algorithm

- KNN uses distance metrics to identify nearest Neighbor, these neighbors are used for classification and regression task.
- To identify nearest neighbor we upcoming distance metrics:

Euclidean Distance

- Euclidean distance is defined as the straight-line distance between two points in a plane or space.
- You can think of it like the shortest path you would walk if you were to go directly from one point to another.

$$distance(x, X_i) = \sqrt{\sum_{j=1}^{d} \left(x_j - X_{i_j}\right)^2}$$

Manhattan Distance

- This is the total distance you would travel if you could only move along horizontal and vertical lines like a grid or city streets.
- It's also called "taxicab distance" because a taxi can only drive along the grid-like streets of a city.

$$d(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$

Minkowski Distance

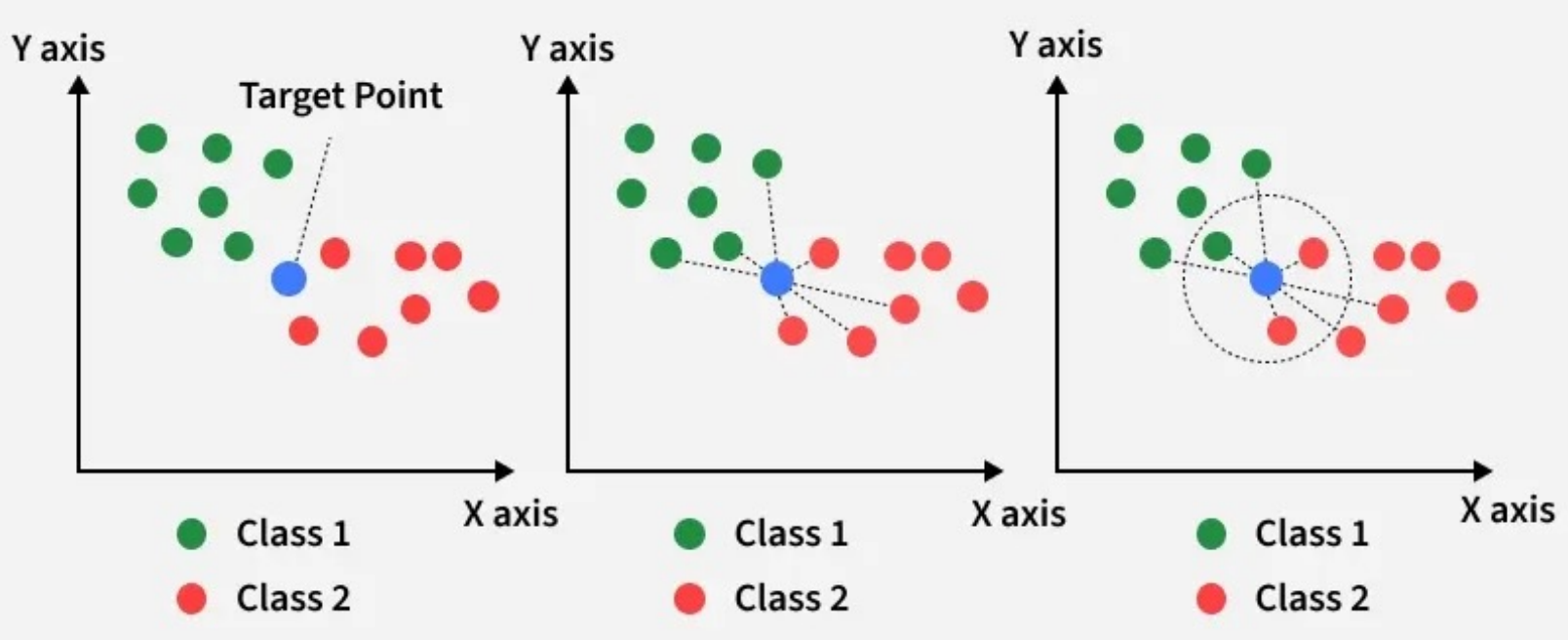
- Minkowski distance is like a family of distances, which includes both Euclidean and Manhattan distances as special cases.

$$d(x, y) = \left(\sum_{i=1}^{n} (x_i - y_i)^p \right)^{\frac{1}{p}}$$

- From the formula above, when $p = 2$, it becomes the same as the Euclidean distance formula and when $p = 1$, it turns into the Manhattan distance formula.
- Minkowski distance is essentially a flexible formula that can represent either Euclidean or Manhattan distance depending on the value of p.

Working of KNN algorithm

- The KNN algorithm operates on the principle of similarity where it predicts the label or value of a new data point by considering the labels or values of its K nearest neighbors in the training dataset.



Step 1: Selecting the optimal value of K

- K represents the number of nearest neighbors that needs to be considered while making prediction.

Step 2: Calculating distance

- To measure the similarity between target and training data points Euclidean distance is widely used.
- Distance is calculated between data points in the dataset and target point.

Step 3: Finding Nearest Neighbors

- The *K* data points with the smallest distances to the target point are nearest neighbors.

Step 4: Voting for Classification

- When you want to classify a data point into a category like spam or not spam, the KNN algorithm looks at the K closest points in the dataset.
- These closest points are called neighbors.
- The algorithm then looks at which category the neighbors belong to and picks the one that appears the most. This is called majority voting.

Step 4: Taking Average for Regression

- In regression, the algorithm still looks for the *K* closest points.
- But instead of voting for a class in classification, it takes the average of the values of those *K* neighbors.
- This average is the predicted value for the new point for the algorithm.

Applications of KNN

- **Recommendation Systems:** Suggests items like movies or products by finding users with similar preferences.
- **Spam Detection:** Identifies spam emails by comparing new emails to known spam and non-spam examples.
- **Customer Segmentation:** Groups customers by comparing their shopping behaviour to others.
- **Speech Recognition:** Matches spoken words to known patterns to convert them into text.

Advantages of KNN

- **Simple to use:** Easy to understand and implement.
- **No training step:** No need to train as it just stores the data and uses it during prediction.
- **Few parameters:** Only needs to set the number of neighbors *K* and a distance method.
- **Versatile:** Works for both classification and regression problems.

Disadvantages of KNN

- **Slow with large data:** Needs to compare every point during prediction.
- **Struggles with many features:** Accuracy drops when data has too many features.
- **Can Overfit:** It can overfit especially when the data is high-dimensional or not clean.

Thank You!